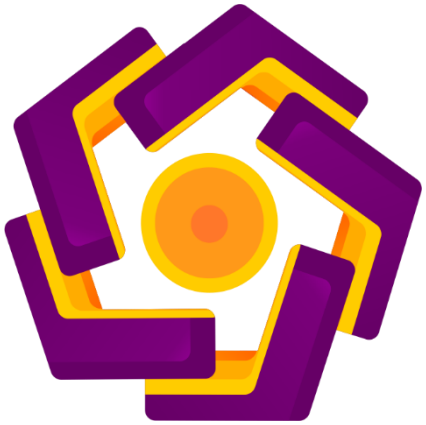
TUGAS 2 KELOMPOK ADVANCED ARITIFICIAL INTELLIGENCE



Disusun oleh:

Muhammad Abiyyu Rahman (25.51.1738)

Muhammad Gufron Hawaly (25.51.1735)

PROGRAM MAGISTER

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI TEKNIK INFORMATIKA

KONSENTRASI DIGITAL TRANSFORMATION INTELLIGENCE

UNIVERSITAS AMIKOM YOGYAKARTA

YOGYAKARTA

2025/2026

Proses Pre-Processing Data

Pada proses Pre-Processing Data melalui langkah sebagai berikut :

1. Ambil Dataset Laptop

Dalam tahap ini, kami mengambil dataset laptop melalui web Kaggle.

Link: https://www.kaggle.com/datasets/jacksondivakarr/laptop-price-prediction-dataset?select=data.csv

2. Pemuatan dan Pembersihan Data

Langkah pertama dalam proses ini merupakan tahapan pemuatan dan pembersihan data, yang bertujuan untuk memastikan bahwa dataset yang akan digunakan benar-benar sudah siap dan bersih sebelum dilakukan proses analisis ataupun pelatihan model kecerdasan buatan. Pada tahap ini, dataset utama yang bernama Laptop.csv dibaca dari lokasi penyimpanan lokal menggunakan library pandas dengan metode read_csv(). Proses pembacaan menggunakan pengaturan encoding='latin1' agar sistem mampu membaca berbagai karakter asing atau simbol yang mungkin terdapat di dalam dataset, mengingat data laptop biasanya diambil dari berbagai sumber internasional yang menggunakan beragam format teks. Setelah data berhasil dimuat, langkah berikutnya adalah menghapus kolom yang tidak memiliki pengaruh terhadap analisis maupun model, yaitu kolom laptop_ID dan Product. Kedua kolom ini bersifat identifikasi semata dan tidak memiliki relevansi dalam menentukan kategori harga ataupun jenis laptop. Selanjutnya, dilakukan pembersihan pada kolom numerik seperti Ram dan Weight yang semula mengandung satuan seperti “GB” dan “kg”. Melalui metode string replacement dan konversi tipe data ke float, kedua kolom tersebut diubah menjadi nilai numerik murni sehingga bisa diolah secara matematis oleh algoritma machine learning. Secara keseluruhan, tahap ini memastikan bahwa data sudah dalam kondisi bersih, bebas dari nilai non-numerik yang tidak dibutuhkan, serta siap untuk diproses lebih lanjut dalam tahap rekayasa fitur.

D: > Test > preprocessinglaptop.py > ...

```python
import pandas as pd
import numpy as np
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

path_lengkap = r"D:\Test\Laptop.csv"
output_dir = r"D:\Test\Processed"
os.makedirs(output_dir, exist_ok=True)

print(f"Membaca file dari: {path_lengkap}")
print(f"Hasil akan disimpan di: {output_dir}\n")
```

```
try:

    data = pd.read_csv(path_lengkap, encoding='latin1')
    data = data.drop(['laptop_ID', 'Product'], axis=1)
    data['Ram'] = data['Ram'].str.replace('GB', '').astype(float)
    data['Weight'] = data['Weight'].str.replace('kg', '').astype(float)

    print("✅ Langkah 1: Pemuatan & Pembersihan Data Selesai")

except FileNotFoundError:
    print(f"❌ ERROR: File tidak ditemukan di {path_lengkap}")
    print("Pastikan Anda telah mengubah variabel 'path_lengkap' sesuai lokasi file Anda.")
    exit()
except Exception as e:
    print(f"❌ ERROR saat memuat data: {e}")
    exit()
```

3. Langkah 2: Rekayasa Fitur (Feature Engineering)

Tahap kedua merupakan proses rekayasa fitur atau **feature engineering**, yaitu upaya untuk mengekstraksi, memodifikasi, dan menambah fitur-fitur baru dari dataset agar model kecerdasan buatan dapat memahami karakteristik laptop dengan lebih baik. Proses ini dilakukan dengan mengubah data mentah menjadi representasi yang lebih informatif dan relevan bagi model prediktif. Langkah pertama dalam tahap ini adalah membuat label klasifikasi harga berdasarkan nilai Price_in_euros. Nilai harga laptop dibagi ke dalam tiga kelompok, yaitu *Murah*, *Sedang*, dan *Mahal*, menggunakan fungsi pd.cut() dengan interval harga tertentu. Pembagian ini sangat penting karena membentuk target prediksi yang konkret bagi model klasifikasi. Selanjutnya, dilakukan analisis terhadap kolom ScreenResolution untuk mengekstraksi dua fitur penting: Touchscreen dan IPS. Fitur ini bernilai 1 jika laptop memiliki layar sentuh atau panel IPS, dan 0 jika tidak. Penambahan fitur ini memungkinkan sistem memahami perbedaan kualitas tampilan antar laptop. Kemudian, kolom Cpu dan Gpu dianalisis untuk mengekstraksi merek prosesor dan kartu grafis utama yang digunakan, karena merek perangkat keras sering kali menjadi indikator kuat terhadap performa dan harga laptop. Nama merek yang jarang muncul digabung ke dalam kategori “Other” agar distribusi data tetap seimbang. Tahapan berikutnya adalah melakukan dekomposisi pada kolom Memory untuk mengidentifikasi tipe penyimpanan yang digunakan oleh laptop, seperti SSD, HDD, Hybrid, atau Flash Storage. Setiap tipe penyimpanan diubah menjadi fitur biner yang menunjukkan keberadaannya dalam setiap unit laptop. Dengan demikian, tahap rekayasa fitur ini berhasil memperkaya dataset dengan atribut baru yang lebih deskriptif, memungkinkan model AI untuk memahami hubungan antara spesifikasi teknis dan kategori harga laptop secara lebih akurat.

```
bins = [0, 700, 1200, data['Price_in_euros'].max()]
labels = ['Murah', 'Sedang', 'Mahal']
data['PriceCategory'] = pd.cut(data['Price_in_euros'], bins=bins, labels=labels, right=False)


data['Touchscreen'] = data['ScreenResolution'].apply(lambda x: 1 if 'Touchscreen' in x else 0)
data['IPS'] = data['ScreenResolution'].apply(lambda x: 1 if 'IPS Panel' in x else 0)

data['Cpu_brand'] = data['Cpu'].apply(lambda x: x.split()[0])

cpu_counts = data['Cpu_brand'].value_counts()
rare_cpus = cpu_counts[cpu_counts <= 10].index
data['Cpu_brand'] = data['Cpu_brand'].replace(rare_cpus, 'Other')

data['Gpu_brand'] = data['Gpu'].apply(lambda x: x.split()[0])
gpu_counts = data['Gpu_brand'].value_counts()
rare_gpus = gpu_counts[gpu_counts <= 10].index
data['Gpu_brand'] = data['Gpu_brand'].replace(rare_gpus, 'Other')

data['SSD'] = data['Memory'].apply(lambda x: 1 if 'SSD' in x else 0)
data['HDD'] = data['Memory'].apply(lambda x: 1 if 'HDD' in x else 0)
data['Flash_Storage'] = data['Memory'].apply(lambda x: 1 if 'Flash Storage' in x else 0)
data['Hybrid'] = data['Memory'].apply(lambda x: 1 if 'Hybrid' in x else 0)

print("✅ Langkah 2: Rekayasa Fitur Selesai")
print(f"   -> Fitur baru dibuat: PriceCategory, Touchscreen, IPS, Cpu_brand, Gpu_brand, SSD, HDD, dll.")
```

4. Langkah 3: Pemisahan Fitur (X) dan Target (y)

Setelah data dibersihkan dan direkayasa, tahap berikutnya adalah pemisahan antara **fitur (X)** sebagai input bagi model dan **target (y)** sebagai label atau output yang ingin diprediksi. Pada tahap ini, hanya kolom yang dianggap relevan dan representatif yang dipilih sebagai fitur utama. Kolom-kolom tersebut meliputi atribut seperti Company, Inches, Ram, Weight, dan OpSys, yang merupakan variabel deskriptif dari laptop, serta fitur baru hasil rekayasa seperti Touchscreen, IPS, Cpu_brand, Gpu_brand, SSD, HDD, Flash_Storage, dan Hybrid. Semua kolom ini digabungkan untuk membentuk satu set fitur komprehensif yang menggambarkan karakteristik fungsional dan performa setiap laptop. Sementara itu, dua target berbeda ditetapkan untuk dua model pembelajaran mesin yang akan dikembangkan, yaitu TypeName untuk klasifikasi jenis laptop (misalnya Ultrabook, Gaming, atau Notebook) dan PriceCategory untuk klasifikasi tingkat harga. Langkah ini juga mencakup pembersihan lanjutan terhadap data target agar tidak ada nilai kosong (NaN) yang bisa mengganggu proses pembelajaran. Dengan memisahkan input dan output secara jelas, sistem memastikan bahwa proses training model nantinya akan berjalan dengan benar dan hasil yang diperoleh benar-benar mencerminkan hubungan sebab-akibat antara spesifikasi laptop dan kategori keluarannya.

```python
feature_columns = [
    'Company', 'Inches', 'Ram', 'Weight', 'OpSys',
    'Touchscreen', 'IPS', 'Cpu_brand', 'Gpu_brand',
    'SSD', 'HDD', 'Flash_Storage', 'Hybrid'
]

X = data[feature_columns]


y_type = data['TypeName']
y_price = data['PriceCategory']


if y_price.isnull().any():
    print("   -> Menghapus baris dengan 'PriceCategory' NaN (mungkin karena harga = 0)")
    nan_indices = y_price[y_price.isnull()].index
    X = X.drop(nan_indices)
    y_type = y_type.drop(nan_indices)
    y_price = y_price.drop(nan_indices)

print("✅ Langkah 3: Pemisahan Fitur (X) dan Target (y) Selesai")
```

5. Langkah 4: Train / Test Split

Tahapan keempat adalah **pemisahan data menjadi set pelatihan (train set)** dan **set pengujian (test set)**, yang merupakan langkah fundamental dalam proses pembelajaran mesin. Tujuannya adalah agar model yang dilatih dapat dievaluasi pada data yang belum pernah dilihat sebelumnya, sehingga performanya dapat diukur secara objektif. Pada skrip ini, fungsi train_test_split() dari pustaka Scikit-learn digunakan untuk membagi data dengan proporsi 80% untuk pelatihan dan 20% untuk pengujian. Pembagian dilakukan dengan parameter random_state=42 untuk menjaga reprodusibilitas hasil, dan stratify=y_type untuk memastikan proporsi setiap kelas jenis laptop tetap seimbang antara data pelatihan dan pengujian. Setelah fitur dan target pertama (TypeName) dibagi, label target kedua (PriceCategory) diambil berdasarkan indeks yang sama agar keselarasan antar data tetap terjaga. Strategi ini penting karena memastikan setiap baris pada X_train memiliki pasangan yang identik dengan label harga dan tipe laptop yang sesuai. Hasilnya adalah dua kumpulan data yang siap digunakan untuk proses pembelajaran, di mana model dapat mempelajari pola dari data pelatihan dan kemudian diuji kemampuannya dalam memprediksi kategori pada data uji yang baru.

```
X_train, X_test, y_train_t, y_test_t = train_test_split(
    X, y_type,
    test_size=0.2,
    random_state=42,
    stratify=y_type
)


y_train_p = y_price.loc[X_train.index]
y_test_p = y_price.loc[X_test.index]

print("✅ Langkah 4: Train/Test Split Selesai")
print(f"   -> Ukuran X_train: {X_train.shape}")
print(f"   -> Ukuran X_test: {X_test.shape}")
```

6. Langkah 5: Preprocessing Pipeline (Scaling & Encoding)

Langkah kelima merupakan inti dari proses **preprocessing pipeline**, yang berfungsi menormalkan data numerik dan mengubah data kategorikal menjadi format numerik agar bisa dipahami oleh algoritma kecerdasan buatan. Tahapan ini dimulai dengan mengidentifikasi tipe data pada setiap kolom fitur. Kolom numerik seperti Ram, Weight, atau Inches akan mengalami proses **standardisasi** menggunakan StandardScaler, di mana setiap nilai dikonversi sehingga memiliki rata-rata nol dan deviasi standar satu. Proses ini penting untuk mencegah perbedaan skala antar fitur memengaruhi performa model, terutama pada algoritma yang sensitif terhadap perbedaan nilai. Sementara itu, kolom kategorikal seperti Company, OpSys, Cpu_brand, dan Gpu_brand akan diubah menjadi representasi numerik biner melalui **One-Hot Encoding** menggunakan OneHotEncoder. Teknik ini menghasilkan kolom baru untuk setiap kategori unik, dengan nilai 1 menandakan keberadaan kategori tersebut pada suatu entri. Untuk menggabungkan kedua proses ini secara bersamaan, digunakan ColumnTransformer, yang memungkinkan penanganan fitur numerik dan kategorikal dalam satu pipeline yang efisien. Setelah pipeline terbentuk, data pelatihan diolah menggunakan fit_transform() agar sistem belajar skala dan struktur kategori dari data pelatihan, sementara data uji hanya melalui transform() agar tidak terjadi kebocoran informasi. Hasil akhir berupa matriks numerik dengan skala seragam dan representasi kategori yang siap digunakan dalam pelatihan model machine learning.

```
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()

print(f"\n   -> Kolom Numerik untuk di-Scale: {numerical_cols}")
print(f"   -> Kolom Kategorikal untuk di-Encode: {categorical_cols}")

numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ],
    remainder='passthrough'
)

X_train_processed_np = preprocessor.fit_transform(X_train)

X_test_processed_np = preprocessor.transform(X_test)

ohe_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_cols)

all_feature_names = numerical_cols + list(ohe_feature_names)

X_train_processed = pd.DataFrame(X_train_processed_np, columns=all_feature_names, index=X_train.index)
X_test_processed = pd.DataFrame(X_test_processed_np, columns=all_feature_names, index=X_test.index)

print("\n✅ Langkah 5: Scaling & Encoding Selesai (via ColumnTransformer)")
print(f"   -> Fitur baru hasil encoding: {len(ohe_feature_names)} kolom")
print(f"   -> Total fitur untuk model: {len(all_feature_names)} kolom")
```

7. Langkah 6: Hasil Processing dan Penyimpanan Data

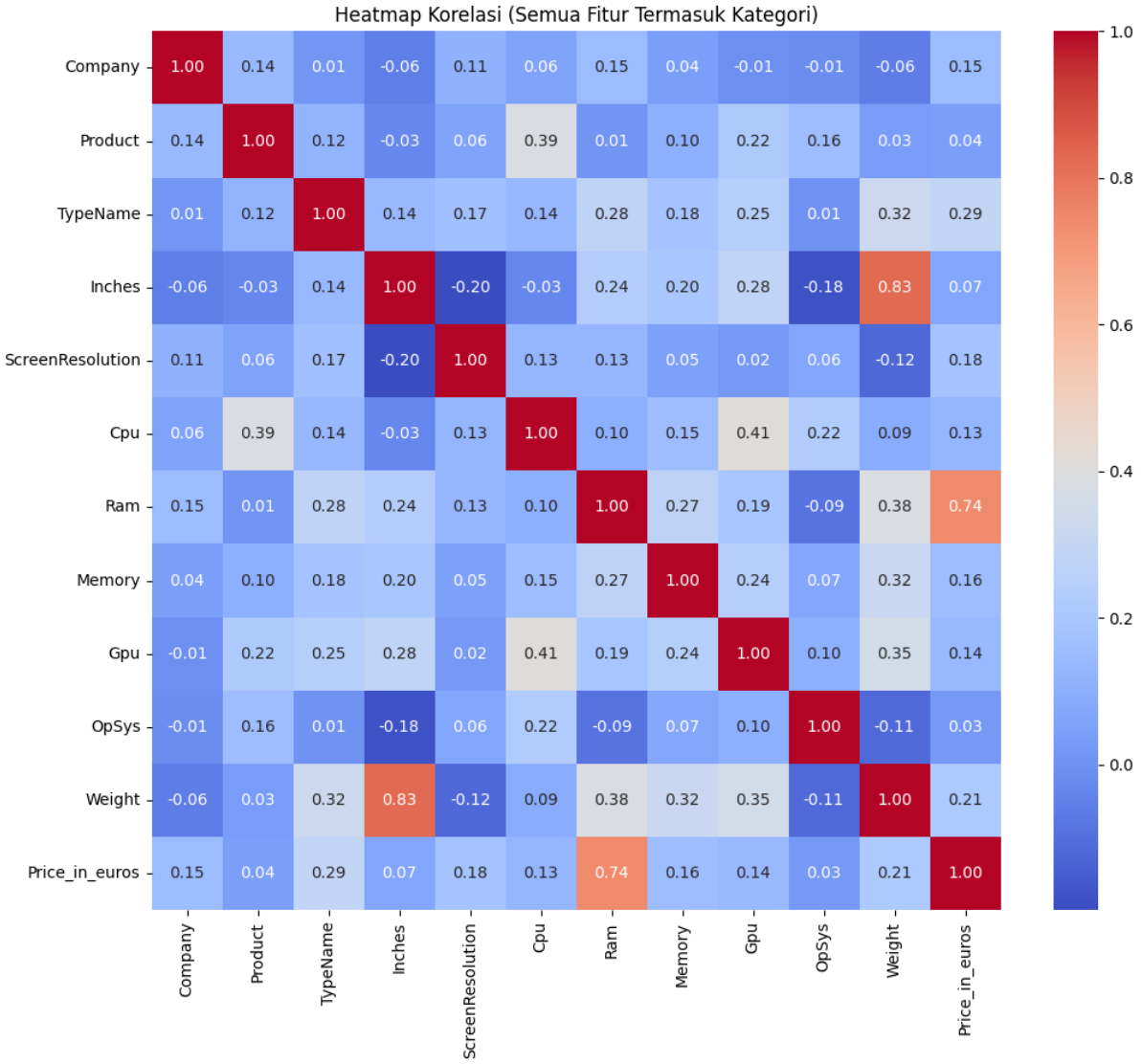
Langkah terakhir dalam keseluruhan proses adalah menyimpan hasil preprocessing ke dalam berkas CSV agar dapat digunakan kembali dalam tahap berikutnya, yaitu pelatihan model kecerdasan buatan. Semua data hasil pengolahan disimpan dalam folder keluaran yang telah ditentukan, dengan struktur penyimpanan yang sistematis untuk memisahkan antara data pelatihan dan data pengujian. File yang dihasilkan terdiri dari enam bagian utama: X_train_processed.csv dan X_test_processed.csv yang berisi fitur hasil scaling dan encoding, y_train_typename.csv dan y_test_typename.csv untuk target prediksi jenis laptop, serta y_train_price.csv dan y_test_price.csv untuk target kategori harga. Penyimpanan dilakukan menggunakan fungsi to_csv() dari Pandas dengan opsi index=False untuk menjaga format tetap bersih. Proses ini memastikan bahwa seluruh hasil preprocessing dapat dimanfaatkan oleh berbagai algoritma machine learning tanpa perlu mengulang tahapan pembersihan data. Dengan demikian, data yang sebelumnya mentah kini telah melalui serangkaian proses transformasi menjadi dataset siap pakai yang memiliki integritas tinggi, konsistensi skala, dan format yang sesuai untuk kebutuhan analisis maupun pelatihan model. Tahapan ini menjadi fondasi penting dalam pembangunan sistem berbasis pengetahuan yang diharapkan mampu memberikan rekomendasi laptop yang optimal dan sesuai kebutuhan pengguna.

```
X_train_processed.to_csv(os.path.join(output_dir, "X_train_processed.csv"), index=False)
X_test_processed.to_csv(os.path.join(output_dir, "X_test_processed.csv"), index=False)
y_train_t.to_csv(os.path.join(output_dir, "y_train_typename.csv"), index=False, header=True)
y_test_t.to_csv(os.path.join(output_dir, "y_test_typename.csv"), index=False, header=True)

y_train_p.to_csv(os.path.join(output_dir, "y_train_price.csv"), index=False, header=True)
y_test_p.to_csv(os.path.join(output_dir, "y_test_price.csv"), index=False, header=True)


print("\n---  SEMUA PROSES SELESAI  ---")
print(f"File-file yang siap untuk modeling telah disimpan di folder: {output_dir}")
print("File yang disimpan:")
print("- X_train_processed.csv (Fitur training, siap untuk model 1 & 2)")
print("- X_test_processed.csv (Fitur testing, siap untuk model 1 & 2)")
print("- y_train_typename.csv (Target 1 - training)")
print("- y_test_typename.csv (Target 1 - testing)")
print("- y_train_price.csv (Target 2 - training)")
print("- y_test_price.csv (Target 2 - testing)")
```

8. EDA (Exploratory Data Analysis) dan Heatmap Korelasi



Gambar diatas merupakan Tabel Heatmap Korelasi antar fitur pada dataset laptop yang kami ambil dari Kaggle. Disini kami menentukan nilai threshold yaitu ≥ 0.3 untuk mengantisipasi terjadinya data teredudansi dan kami ingin proses selanjutnya sesuai dengan sistem rekomendasi laptop "berdasarkan harga dan kebutuhan pengguna".

Korelasi dari Heatmap diatas juga digunakan bukan untuk prediksi harga, tapi untuk:

1. Menentukan fitur relevan yang menggambarkan *kemiripan antar laptop* (misalnya RAM, CPU, GPU, dan harga).

2. Menghindari fitur yang tidak berpengaruh (misalnya "Company" atau "Operating System" jika tidak relevan dengan kebutuhan pengguna).

Berdasarkan Heatmap Korelasi yang sudah ditentukan nilai thresholdnya yaitu 0.3 maka fitur yang digunakan untuk tahapan selanjutnya yaitu:

1. **RAM** dengan nilai korelasi 0.74 (kuat sekali, harga meningkat seiring kapasitas RAM).

2 .**GPU** dengan nilai korelasi 0.35 (cukup berpengaruh, laptop dengan GPU bagus cenderung lebih mahal).

10. Hasil dan Kesimpulan

Setelah melalui proses diatas, dataset yang diambil dari Kaggle sebelumnya sudah bersih, tidak ada nilai kosong atau duplikat, dan semua fitur sudah siap untuk diproses oleh model AI karena seluruh fitur sudah berbentuk numerik, tidak ada nilai hilang, dan fitur relevan telah dipilih sehingga dapat masuk ke proses atau tahapan selanjutnya yaitu pemodelan. Berikut link CSV hasil pre-processing dataset:

Link Hasil : https://drive.google.com/drive/folders/1jjTdicJVjznTxdW-Gq5F_hJyL8qIQAmg?usp=drive_link